

Multi-Insert in Logical Replication

Batching the Apply Worker for Fun and Throughput

Andrei Lepikhov

pgEdge

09.05.2026

Why This Matters

1. Async replication needs minimal lag.

Logical replication is asynchronous – every second of apply delay is a second of data you cannot read on the subscriber. Reducing per-tuple overhead shrinks that window.

2. Bulk influx is the typical pattern.

End-of-month batch loads, nightly ETL jobs – real workloads frequently arrive in bursts, not drips. The apply worker must handle millions of INSERTs at wire speed.

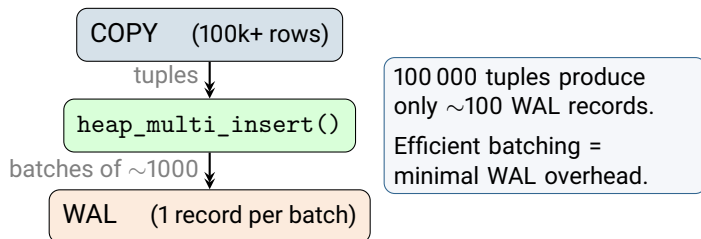
3. Multi-region means low-bandwidth channels.

Multi-cloud and cross-continent logical replication runs over constrained links. Every extra byte of WAL on the subscriber is wasted disk I/O *and* wasted recovery time when the link catches up.

Today the subscriber can write significantly more WAL than the publisher for the same COPY – pure overhead.

The Problem

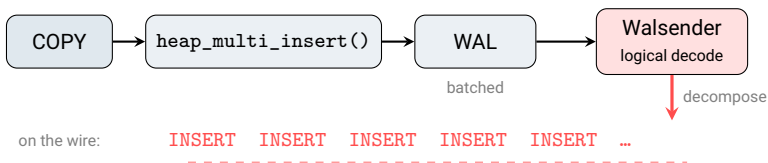
How COPY Loads Data on the Publisher



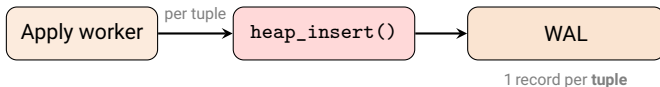
But what happens on the **subscriber**?

Without Batching, Subscriber WAL Exceeds Publisher

Publisher



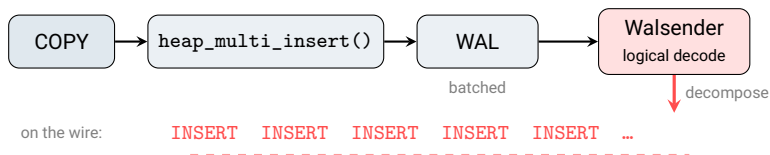
Subscriber



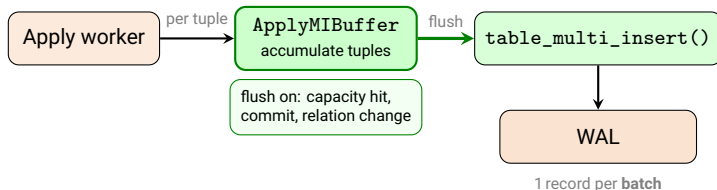
Publisher COPY writes 13 GB of WAL $\Rightarrow$ subscriber generates **~18 GB**.
WAL amplification on subscriber. Slow catch-up. Wasted I/O.

The Fix: Batch Inserts in the Apply Worker

Publisher



Subscriber



Why No MULTI_INSERT in the community Protocol?

- **Not a SQL operation.**

Protocol mirrors SQL changes; `heap_multi_insert()` is internal to COPY.

- **Protocol version negotiation.**

New message type $\Rightarrow$ version bump, fallback paths, coordination cost.

- **Conflict resolution becomes harder.**

Batch is all-or-nothing — must unpack and retry row-by-row on conflict.

- **Output plugin API is per-change.**

`LogicalDecodeChangeCB` receives one change — batch API breaks all existing plugins.

Enabling Multi-Insert

-- Automatic slot creation:

```
CREATE SUBSCRIPTION mysub  
  CONNECTION 'host=publisher port=5432 dbname=mydb'  
  PUBLICATION mypub  
  WITH (multi_insert = on, streaming = off);
```

-- With a manually created slot:

-- 1. On publisher (replication connection):

```
CREATE_REPLICATION_SLOT mysub_slot LOGICAL pgoutput;
```

-- 2. On subscriber:

```
CREATE SUBSCRIPTION mysub  
  CONNECTION 'host=publisher port=5432 dbname=mydb'  
  PUBLICATION mypub  
  WITH (  
    create_slot = false,  
    slot_name = 'mysub_slot',  
    multi_insert = on,  
    streaming = off  
  );
```

`multi_insert` is always a subscription option – no protocol changes.

Implementation

ApplyMIBuffer – The Batching Engine

Lifecycle:

- 1 Single static buffer, initialized on first INSERT for an eligible relation
- 2 Accumulate tuples in slot array
- 3 **Intermediate flush:** capacity hit $\rightarrow$ call `table_multi_insert()`, clear slots, reuse buffer
- 4 **Final flush:** commit, relation change, or non-INSERT event $\rightarrow$ flush and reinitialize for the next relation

Slot management:

- Fixed array of 10 000 `TupleTableSlot*`, lazily allocated one per tuple
- Slots reused across flushes (only `nslots` resets to 0)
- Flush when `nslots` $\geq$ 10 000 or `cum_bytes` $\geq$ 8 MB

Not Every Table Qualifies for Batching

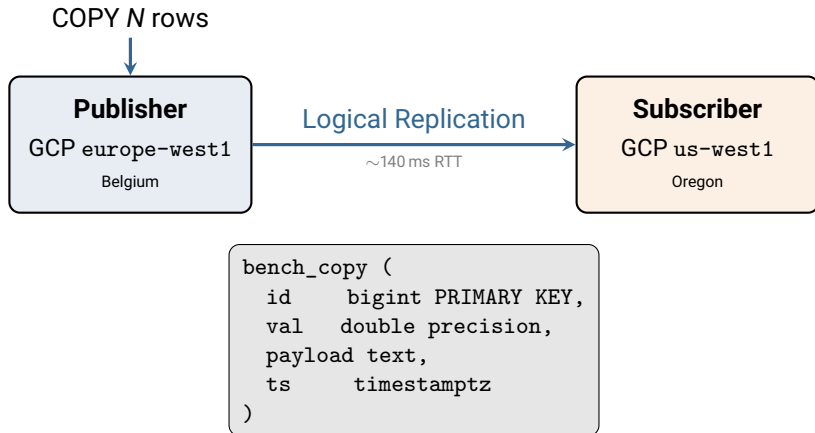
A table is eligible for multi-insert if it has **none** of:

- BEFORE/INSTEAD OF triggers
- Row-level security
- CHECK constraints
- Stored generated columns
- Exclusion constraints
- Deferrable UNIQUE/PK
- Invalid or unready indexes
- Non-plain table (partitioned, foreign)

Ineligible tables **silently fall back** to per-tuple insert.

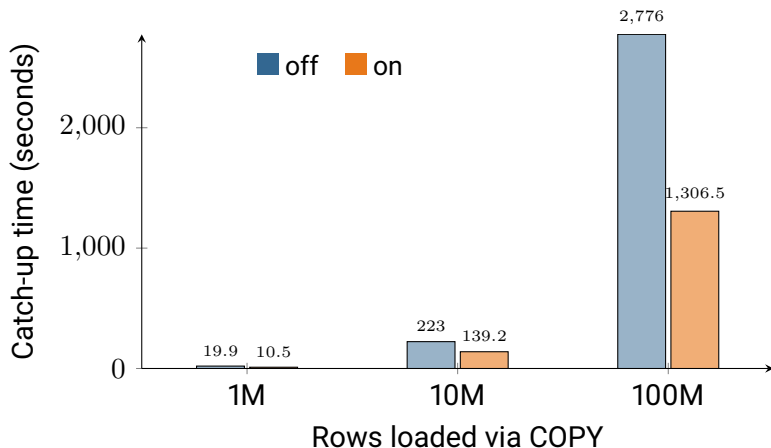
Stricter than COPY: LR bypasses the executor, so constraints that COPY validates at runtime must be rejected statically.

Benchmark Setup



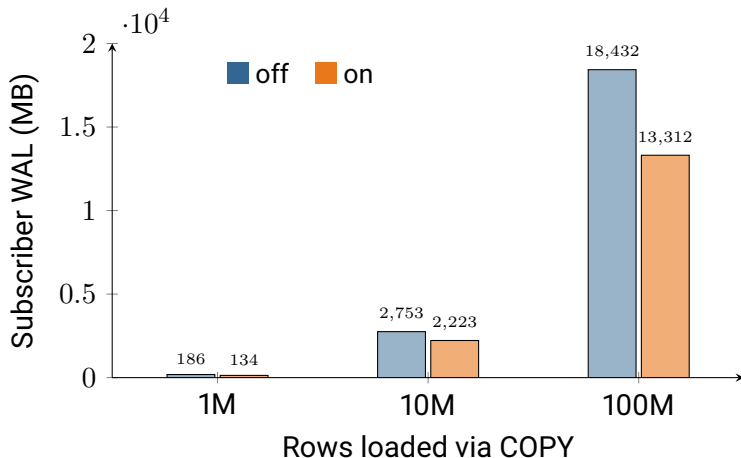
- Three dataset sizes: 1M, 10M, 100M rows
- `multi_insert = true` VS `false`, `streaming = off`

Multi-insert halves replication catch-up



1.6× – 2.1× faster replication catch-up across all sizes.

Multi-insert cuts subscriber WAL by 19–28%



At 100M rows: 18 GB $\rightarrow$ 13 GB.

The Numbers at a Glance

Rows	Catch-up (s)		Subscriber WAL	
	off	on	off	on
1M	19.9	10.5	186 MB	134 MB
10M	223.0	139.2	2.7 GB	2.2 GB
100M	2 776	1 307	18 GB	13 GB
Speedup	1.6–2.1×		–19% to –28%	

Publisher COPY time and publisher WAL are **unchanged** – the optimisation is purely subscriber-side.

Current Limitations and Open Problems

- **Conflicts and errors cause fallback to single-insert.**

A unique violation or other error during batch flush aborts the remote transaction. Standard per-row conflict resolution (`disable_on_error`, `SKIP`) is bypassed.

- **Streaming must incorporate batching.**

Currently `multi_insert` requires `streaming = off`. Combining both is needed for a production-ready version.

- **Parallel apply workers excluded.**

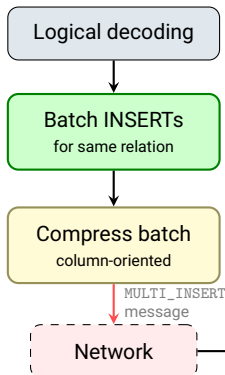
Multi-insert is disabled inside parallel apply workers.

- **Per-transaction disable on first ineligible relation.**

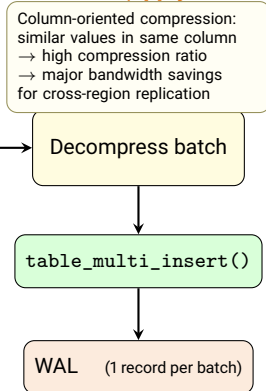
If any relation fails the eligibility check, batching is disabled for the *entire* remaining transaction, even for other eligible tables.

Future: publisher-side batching

Publisher (walsender)



Subscriber (apply worker)



Summary

- Logical replication apply worker now supports batched inserts
- **2× faster** replication catch-up on bulk loads
- **28% less WAL** generated on the subscriber
- Subscriber-side only – zero publisher changes
- Opt-in via `multi_insert = on`, graceful fallback



Code



Benchmarks & materials

Questions?

Publisher Side: Many Writers, Few Senders, One WAL

